Rapport de Projet : Gestion du Parc Informatique et des Incidents (GSB)

1. Introduction et Contexte

1.1 Présentation de l'entreprise

Le laboratoire Galaxy Swiss Bourdin (GSB) est né de la fusion entre le géant américain Galaxy et le conglomérat européen Swiss Bourdin. Le siège administratif européen est situé à Paris. Le système informatique du laboratoire compte plus de 350 équipements terminaux et un nombre croissant de serveurs.

1.2 Objectifs du projet

Afin d'assurer une gestion efficace de ce parc informatique, le laboratoire a souhaité mettre en place une application permettant de gérer le matériel et d'assurer l'assistance en cas de panne. Les objectifs principaux de l'application développée sont:

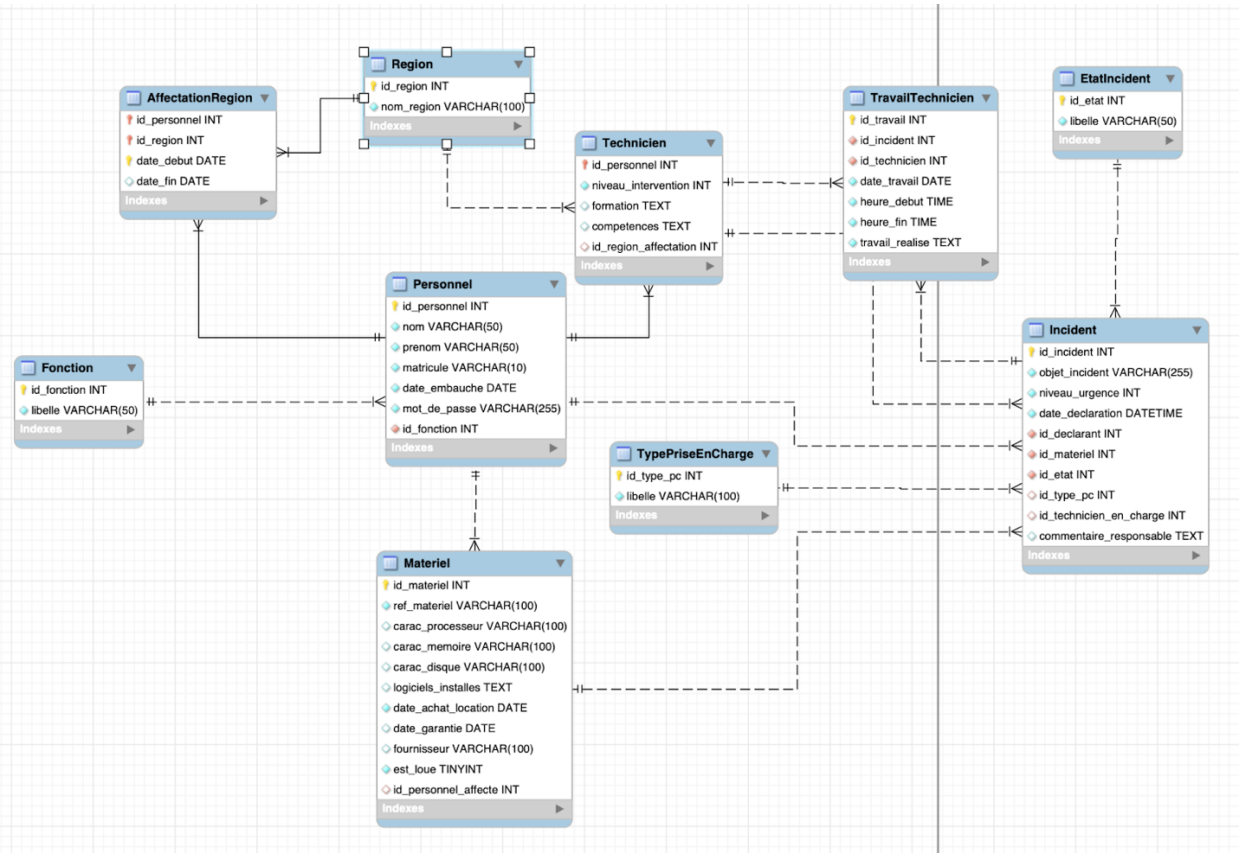
- Gérer l'inventaire du matériel (achat, location, garantie).
- Centraliser la gestion des incidents (tickets).
- Optimiser le suivi par les techniciens et fournir des statistiques aux responsables.

2. Analyse et Conception (Base de Données)

2.1 Modèle Conceptuel de Données (MCD)

Nous avons élaboré un MCD permettant de lier le personnel, le matériel et les incidents. Les entités principales identifiées sont :

- Personnel : Distinction entre Techniciens, Visiteurs et Responsables.
- Matériel : Caractéristiques techniques (processeur, mémoire) et contractuelles.
- Incident : Objet, urgence, état et lien vers le matériel concerné.
- TravailTechnicien : Suivi des interventions (heures, travail réalisé).



2.2 Modèle Logique de Données (MLD)

Suite au passage au relationnel, voici la structure de notre base de données MySQL :

- **Fonction** (id_fonction, libelle)
- **Region** (id_region, nom_region)
- **EtatIncident** (id_etat, libelle)
- **TypePriseEnCharge** (id_type_pc, libelle)
- **Personnel** (id_personnel, nom, prenom, matricule, date_embauche, mot_de_passe, #id_fonction)
- **Technicien** (id_personnel, niveau_intervention, formation, competences, #id_region_affectation)
- **Materiel** (id_materiel, ref_materiel, carac_processeur, ..., #id_personnel_affecte)
- **Incident** (id_incident, objet_incident, niveau_urgence, date_declaration, #id_declarant, #id_materiel, #id_etat, #id_type_pc, #id_technicien_en_charge, commentaire_responsable)
- **TravailTechnicien** (id_travail, #id_incident, #id_technicien, date_travail, heure_debut, heure_fin, travail_realise)

3. Architecture Technique et Choix de Développement

L'application respecte une architecture en couches stricte, séparant la logique de présentation, les objets métier et l'accès aux données, conformément au cahier des charges.

3.1 Les Classes Métier

Les données sont manipulées sous forme d'objets C#. Nous avons créé une classe pour chaque entité de la base de données (ex: Incident.cs, Materiel.cs, Technicien.cs).

- Encapsulation : Utilisation de propriétés (get; set;) pour protéger l'accès aux champs privés.
- Héritage : La classe Technicien hérite de la classe Personnel, permettant de réutiliser les attributs communs (nom, matricule) tout en ajoutant les spécificités (niveau d'intervention, formation).

3.2 La Couche d'Accès aux Données (DAL)

Toute la communication avec la base MySQL est centralisée dans la classe DatabaseConnection.cs.

- Méthodes Statiques : Comme demandé, les méthodes d'accès sont statiques (ex: DatabaseConnection.AjouterIncident(...)), ce qui permet de les appeler depuis n'importe quel formulaire sans instancier la classe de connexion à chaque fois.
- Sécurité : Nous avons systématiquement utilisé des requêtes préparées avec paramètres (@matricule, @nom) pour empêcher les injections SQL.
- Exemple de code sécurisé : command.Parameters.AddWithValue("@matricule", unPersonnel.Matricule);

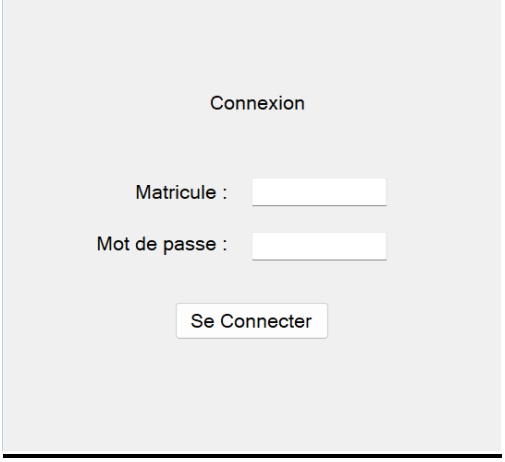
3.3 Interface Utilisateur (WinForms)

L'interface est gérée via des Windows Forms. Aucune requête SQL n'est présente directement dans les formulaires ; ces derniers appellent uniquement les méthodes de DatabaseConnection.

4. Analyse des Fonctionnalités Développées

4.1 Authentification et Gestion des Rôles

Le formulaire LoginForm gère l'entrée dans l'application.

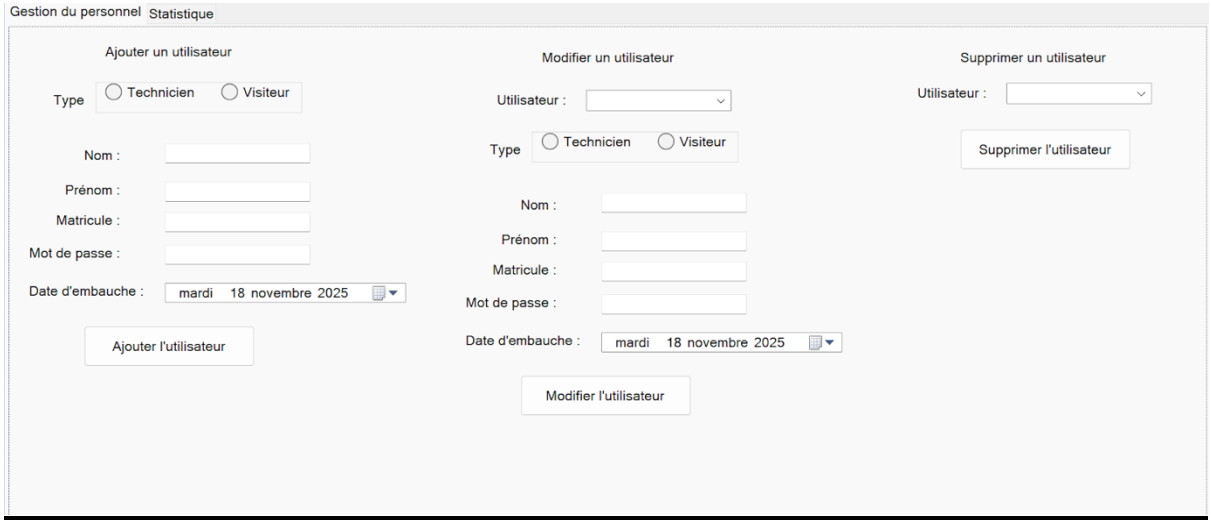


La méthode Connexion vérifie le couple Matricule/Mot de passe et retourne l'ID de la fonction (rôle).

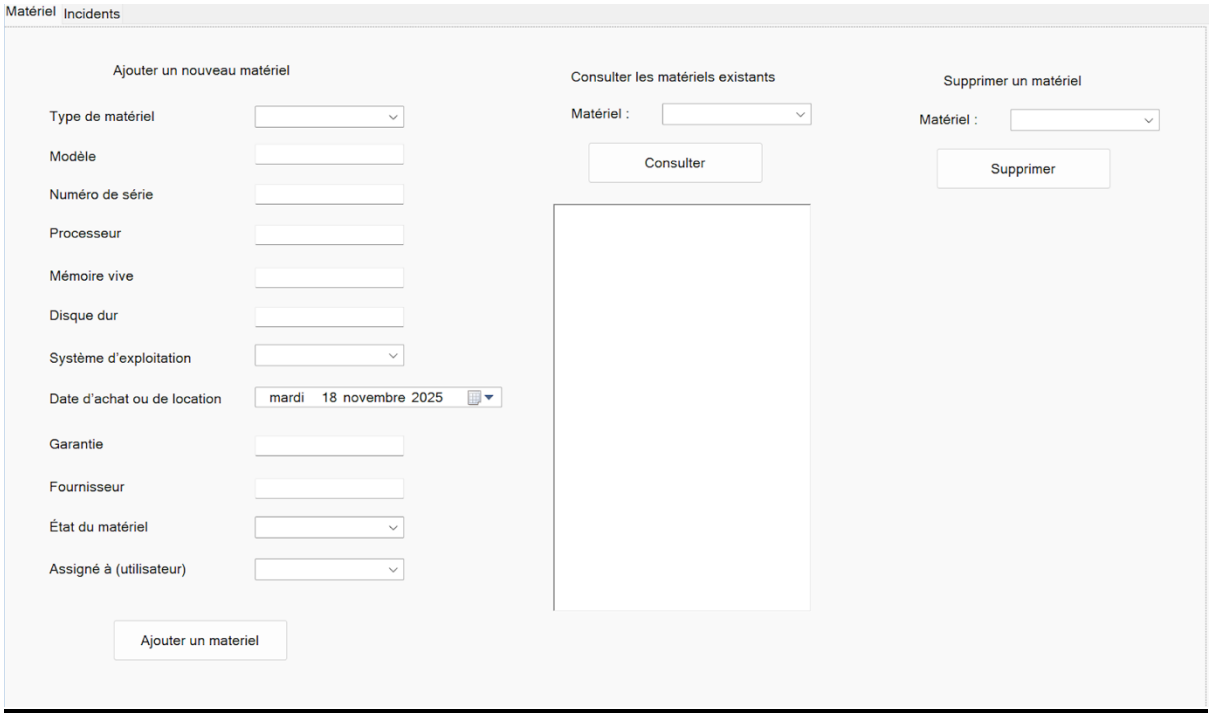
Un switch redirige l'utilisateur vers le formulaire adapté à ses droits :

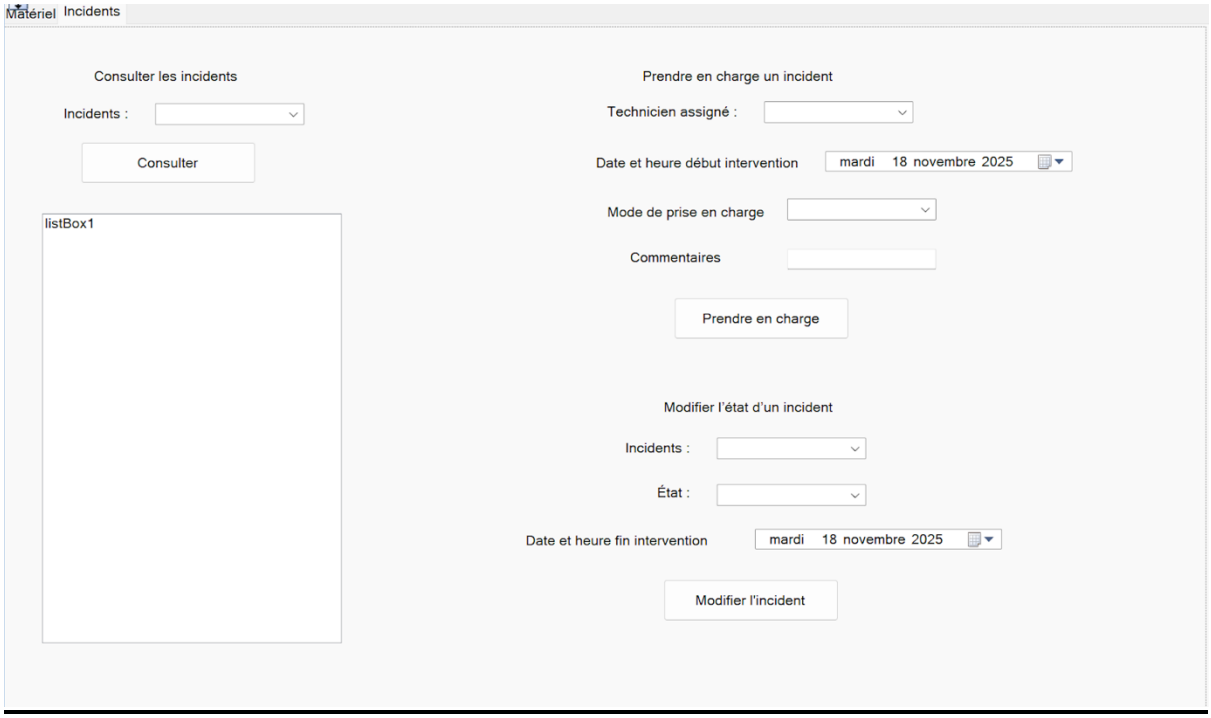
- Rôle 1 : VisiteurForm

- Rôle 2 : ResponsableForm



- Rôle 3 : TechnicienForm





4.2 Module Visiteur

Le formulaire VisiteurForm permet aux employés de :

- Déclarer un incident : L'utilisateur sélectionne son matériel (chargé via GetAllMateriel), définit une urgence et décrit le problème. L'incident est créé avec l'état "Enregistrée" (ID 1) par défaut.
- Consulter son matériel : Une méthode de filtrage Materiel.FilterByPersonnelId permet d'afficher uniquement le matériel assigné à l'utilisateur connecté.

4.3 Module Technicien et Responsable

Bien que l'interface graphique soit prête (TechnicienForm avec des onglets "Matériel" et "Incidents"), la logique métier s'appuie sur les méthodes CRUD robustes implémentées dans DatabaseConnection :

- CRUD Matériel : Ajout, Modification et Suppression de matériel (AjouterMateriel, SupprimerMateriel).
- Gestion des Incidents : Mise à jour de l'état des tickets et ajout des comptes-rendus d'intervention via la table TravailTechnicien.
- Gestion du Personnel (Responsable) : Ajout de nouveaux techniciens ou visiteurs avec gestion automatique des clés étrangères.

5. Bilan du Projet

5.1 Réalisations

- Mise en place réussie d'une base de données relationnelle MySQL.
- Développement d'une application C# fonctionnelle avec gestion des sessions utilisateurs.
- Respect des principes de la programmation orientée objet (Héritage, Encapsulation).
- Code sécurisé contre les failles SQL classiques.

5.2 Améliorations Possibles

- Finaliser le câblage des événements (boutons) sur les formulaires Technicien et Responsable pour exploiter pleinement les méthodes de la classe DatabaseConnection déjà codées.
- Ajouter le hachage des mots de passe (actuellement stockés en clair) pour renforcer la sécurité.

6. **Annexes**

6.1 Classe Metier

Personnel :

```
internal class Personnel
  {
    private int id_personnel;
    private string nom;
    private string prenom;
    private string matricule;
    private DateTime date_embauche;
    private string mot_de_passe;
    private int id_fonction;

    public Personnel(int id_personnel, string nom, string prenom, string matricule,
DateTime date_embauche, string mot_de_passe, int id_fonction)
    {
      this.Id_personnel = id_personnel;
      this.Nom = nom;
      this.Prenom = prenom;
      this.Matricule = matricule;
      this.Date_embauche = date_embauche;
      this.Mot_de_passe = mot_de_passe;
      this.id_fonction = id_fonction;
    }

    public int Id_personnel { get => id_personnel; set => id_personnel = value; }
    public string Nom { get => nom; set => nom = value; }
```

```
    public string Prenom { get => prenom; set => prenom = value; }
    public string Matricule { get => matricule; set => matricule = value; }
    public DateTime Date_embauche { get => date_embauche; set => date_embauche =
value; }
    public string Mot_de_passe { get => mot_de_passe; set => mot_de_passe = value; }
    public int Id_fonction { get => id_fonction; set => id_fonction = value; }
  }
```

Materiel :

```
internal class Materiel
  {
    private int id_materiel;
    private string ref_materiel;
    private string processeur;
    private string memoire;
    private string disque_dur;
    private DateTime date_achat;
    private DateTime garantie;
    private string fournisseur;
    private int id_personnel;

    public int Id_materiel { get => id_materiel; set => id_materiel = value; }
    public string Ref_materiel { get => ref_materiel; set => ref_materiel = value; }
    public string Processeur { get => processeur; set => processeur = value; }
    public string Memoir { get => memoire; set => memoire = value; }
    public string Disque_dur { get => disque_dur; set => disque_dur = value; }
    public DateTime Date_achat { get => date_achat; set => date_achat = value; }
    public DateTime Garantie { get => garantie; set => garantie = value; }
```

```
    public string Fournisseur { get => fournisseur; set => fournisseur = value; }

    public int Id_personnel { get => id_personnel; set => id_personnel = value; }


    public Materiel(int id_materiel, string ref_materiel, string processeur, string memoir,
string disque_dur, DateTime date_achat, DateTime garantie, string fournisseur, int
id_personnel)

    {

        Id_materiel = id_materiel;

        Ref_materiel = ref_materiel;

        Processeur = processeur;

        Memoir = memoir;

        Disque_dur = disque_dur;

        Date_achat = date_achat;

        Garantie = garantie;

        Fournisseur = fournisseur;

        Id_personnel = id_personnel;

    }


    public static List<Materiel> FilterByPersonnelId(List<Materiel> materiels, int
idPersonnel)

    {

        return materiels.Where(m => m.Id_personnel == idPersonnel).ToList();

    }


  }
```

Incidents :

```
internal class Incident

  {
```

```
    private int id_incident;
    private string objet;
    private int niveau_urgence;
    private DateTime date_declaration;
    private int id_declarant;
    private int id_materiel;
    private int etat;
    private int id_type;
    private int id_technicien;

    public int Id_incident { get => id_incident; set => id_incident = value; }
    public string Objet { get => objet; set => objet = value; }
    public int Niveau_urgence { get => niveau_urgence; set => niveau_urgence = value; }
    public DateTime Date_declaration { get => date_declaration; set => date_declaration
= value; }
    public int Id_declarant { get => id_declarant; set => id_declarant = value; }
    public int Id_materiel { get => id_materiel; set => id_materiel = value; }
    public int Etat { get => etat; set => etat = value; }
    public int Id_type { get => id_type; set => id_type = value; }
    public int Id_technicien { get => id_technicien; set => id_technicien = value; }

    public Incident(int id_incident, string objet, int niveau_urgence, DateTime
date_declaration, int id_declarant, int id_materiel, int etat, int id_type, int id_technicien)
    {
        Id_incident = id_incident;
        Objet = objet;
        Niveau_urgence = niveau_urgence;
        Date_declaration = date_declaration;
        Id_declarant = id_declarant;
```

```
        Id_materiel = id_materiel;
        Etat = etat;
        Id_type = id_type;
        Id_technicien = id_technicien;
    }
  }
```

TravailTechnicien :

```
internal class TravailTechnicien
  {
    private int id_travail;
    private int id_incident;
    private int id_technicien;
    private DateTime date_travail;
    private TimeSpan heure_debut;
    private TimeSpan heure_fin;
    private string travail_realise;

    public int Id_travail { get => id_travail; set => id_travail = value; }
    public int Id_incident { get => id_incident; set => id_incident = value; }
    public int Id_technicien { get => id_technicien; set => id_technicien = value; }
    public DateTime Date_travail { get => date_travail; set => date_travail = value; }
    public TimeSpan Heure_debut { get => heure_debut; set => heure_debut = value; }
    public TimeSpan Heure_fin { get => heure_fin; set => heure_fin = value; }
    public string Travail_realise { get => travail_realise; set => travail_realise = value; }

    public TravailTechnicien(int id_travail, int id_incident, int id_technicien, DateTime
date_travail, TimeSpan heure_debut, TimeSpan heure_fin, string travail_realise)
```

```
    {
        Id_travail = id_travail;
        Id_incident = id_incident;
        Id_technicien = id_technicien;
        Date_travail = date_travail;
        Heure_debut = heure_debut;
        Heure_fin = heure_fin;
        Travail_realise = travail_realise;
    }
}
```

Region :

```
internal class Region
{
    public int Id_region { get; set; }
    public string NomRegion { get; set; }
}
```

EtatIncident :

```
internal class EtatIncident
{
    public int Id_etat { get; set; }
    public string Libelle { get; set; }

    // Constructeur
    public EtatIncident(int id, string libelle)
    {
        Id_etat = id;
```

```
        Libelle = libelle;
    }

    // Liste statique des états
    public static readonly List<EtatIncident> Etats = new List<EtatIncident>()
    {
        new EtatIncident(1, "Enregistrée"),
        new EtatIncident(2, "En cours de traitement"),
        new EtatIncident(3, "Résolue")
    };
  }
```

TypePriseEnCharge :

```
internal class TypePriseEnCharge
  {
    public int Id_type_prise_en_charge { get; set; }
    public string Libelle { get; set; }
  }
```

Technicien héritage de Personnel :

```
internal class Technicien:Personnel
{
  private int id_technicien;
  private int niveau_intervention;
  private string formation;
  private string competence;
  private int id_region;
```

```
    public int Id_technicien { get => id_technicien; set => id_technicien = value; }

    public int Niveau_intervention { get => niveau_intervention; set => niveau_intervention
= value; }

    public string Formation { get => formation; set => formation = value; }

    public string Competence { get => competence; set => competence = value; }

    public int Id_region { get => id_region; set => id_region = value; }


    public Technicien(int id_personnel, string nom, string prenom, string matricule,
DateTime date_embauche, string mot_de_passe, int id_fonction,

        int id_technicien, int niveau_intervention, string formation, string competence, int
id_region)

        : base(id_personnel, nom, prenom, matricule, date_embauche, mot_de_passe,
id_fonction)

    {

        Id_technicien = id_technicien;

        Niveau_intervention = niveau_intervention;

        Formation = formation;

        Competence = competence;

        Id_region = id_region;

    }

}
```

TypePc :

```
internal class TypePc

{

    public int Id_type { get; set; }

    public string Libelle { get; set; }
```

```
    public TypePc(int id_type, string libelle)
    {
        Id_type = id_type;
        Libelle = libelle;
    }
    public static readonly List<TypePc> Liste = new List<TypePc>()
    {
        new TypePc(1, "Bureau"),
        new TypePc(2, "Portable")
    };
}
```

6.2 Classe DatabaseConnexion

```
internal class DatabaseConnection
{
    private readonly string _connectionString;
    public DatabaseConnection()
    {
        _connectionString =
"Server=localhost;Database=projetcsharp;Uid=root;Pwd=root;";
    }
    public MySqlConnection GetConnection()
```

```
    {
        return new MySqlConnection(_connectionString);
    }

    public static void AjouterTechnicien(Technicien unTechnicien)
    {
        DatabaseConnection db = new DatabaseConnection();
        using (MySqlConnection connection = db.GetConnection())
        {
            connection.Open();
            string query = "INSERT INTO personnel (nom, prenom, matricule, date_embauche,
mot_de_passe, id_fonction) " +
                        "VALUES (@nom, @prenom, @matricule, @date_embauche,
@mot_de_passe, @id_fonction); " +
                        "SELECT LAST_INSERT_ID();";
            using (MySqlCommand command = new MySqlCommand(query, connection))
            {
                command.Parameters.AddWithValue("@nom", unTechnicien.Nom);
                command.Parameters.AddWithValue("@prenom", unTechnicien.Prenom);
                command.Parameters.AddWithValue("@matricule", unTechnicien.Matricule);
                command.Parameters.AddWithValue("@date_embauche",
unTechnicien.Date_embauche);
                command.Parameters.AddWithValue("@mot_de_passe",
unTechnicien.Mot_de_passe);
                command.Parameters.AddWithValue("@id_fonction", 2);
                // Execute the insert and get the new personnel ID
                int newPersonnelId = Convert.ToInt32(command.ExecuteScalar());
                // Now insert into technicien table
```

```
            string techQuery = "INSERT INTO technicien (id_personnel, niveau_intervention,
formation, competence, id_region) " +
                                "VALUES (@id_personnel, @niveau_intervention, @formation,
@competence, @id_region);";
            using (MySqlCommand techCommand = new MySqlCommand(techQuery,
connection))
            {
                techCommand.Parameters.AddWithValue("@id_personnel", newPersonnelId);
                techCommand.Parameters.AddWithValue("@niveau_intervention",
unTechnicien.Niveau_intervention);
                techCommand.Parameters.AddWithValue("@formation",
unTechnicien.Formation);
                techCommand.Parameters.AddWithValue("@competence",
unTechnicien.Competence);
                techCommand.Parameters.AddWithValue("@id_region",
unTechnicien.Id_region);
                techCommand.ExecuteNonQuery();
            }
        }
    }
}

public static void AjouterVisiteur(Personnel unPersonnel)
{
    DatabaseConnection db = new DatabaseConnection();
    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
        string query = "INSERT INTO personnel (nom, prenom, matricule, date_embauche,
mot_de_passe,id_fonction) " +
```

```
                "VALUES (@nom, @prenom, @matricule, @date_embauche,
@mot_de_passe, @id_fonction);";
        using (MySqlCommand command = new MySqlCommand(query, connection))
        {
            command.Parameters.AddWithValue("@nom", unPersonnel.Nom);
            command.Parameters.AddWithValue("@prenom", unPersonnel.Prenom);
            command.Parameters.AddWithValue("@matricule", unPersonnel.Matricule);
            command.Parameters.AddWithValue("@date_embauche",
unPersonnel.Date_embauche);
            command.Parameters.AddWithValue("@mot_de_passe",
unPersonnel.Mot_de_passe);
            command.Parameters.AddWithValue("@id_fonction", 1);


            command.ExecuteNonQuery();
        }
    }
  }



  public static void AjouterMateriel(Materiel unMateriel)
  {
    DatabaseConnection db = new DatabaseConnection();


    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();


        string query = "INSERT INTO materiel (ref_materiel, carac_processeur,
carac_memoire, carac_disque, " +
```

```
                "date_achat_location, date_garantie, fournisseur, id_personnel_affecte) " +
                "VALUES (@ref_materiel, @processeur, @memoire, @disque,
@date_achat, @garantie, @fournisseur, @id_personnel);";

        using (MySqlCommand command = new MySqlCommand(query, connection))
        {
            command.Parameters.AddWithValue("@ref_materiel",
unMateriel.Ref_materiel);
            command.Parameters.AddWithValue("@processeur", unMateriel.Processeur);
            command.Parameters.AddWithValue("@memoire", unMateriel.Memoir);
            command.Parameters.AddWithValue("@disque", unMateriel.Disque_dur);
            command.Parameters.AddWithValue("@date_achat", unMateriel.Date_achat);
            command.Parameters.AddWithValue("@garantie", unMateriel.Garantie);
            command.Parameters.AddWithValue("@fournisseur", unMateriel.Fournisseur);
            command.Parameters.AddWithValue("@id_personnel",
unMateriel.Id_personnel);

            command.ExecuteNonQuery();
        }
    }
  }

  public static void AjouterIncident(Incident unIncident)
  {
    DatabaseConnection db = new DatabaseConnection();
    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
```

```
        string query = "INSERT INTO
incident(objet_incident,niveau_urgence,date_declaration,id_declarant,id_materiel,id_e
tat,id_type_pc,id_technicien_en_charge,commentaire_responsable)"+

                    " VALUES
(@objet_incident,@niveau_urgence,@date_declaration,@id_declarant,@id_materiel,@i
d_etat,@id_type_pc,@id_technicien_en_charge,@commentaire_responsable);";

        using (MySqlCommand command = new MySqlCommand(query, connection))

        {

            command.Parameters.AddWithValue("@objet_incident", unIncident.Objet);

            command.Parameters.AddWithValue("@niveau_urgence",
unIncident.Niveau_urgence);

            command.Parameters.AddWithValue("@date_declaration",
unIncident.Date_declaration);

            command.Parameters.AddWithValue("@id_declarant",
unIncident.Id_declarant);

            command.Parameters.AddWithValue("@id_materiel", unIncident.Id_materiel);

            command.Parameters.AddWithValue("@id_etat", unIncident.Etat);

            command.Parameters.AddWithValue("@id_type_pc", unIncident.Id_type);

            command.Parameters.AddWithValue("@id_technicien_en_charge",
unIncident.Id_technicien);

            command.Parameters.AddWithValue("@commentaire_responsable", "");


            command.ExecuteNonQuery();

        }

    }

  }


  public static void AjouterTravailTechnicien(TravailTechnicien unTravailTechnicien)

  {

    DatabaseConnection db = new DatabaseConnection();
```

```
    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
        string query = "INSERT INTO travailtechnicien (id_incident, id_technicien,
date_travail, heure_debut, heure_fin, travail_realise) " +
                    "VALUES (@id_incident, @id_technicien, @date_travail, @heure_debut,
@heure_fin, @travail_realise);";
        using (MySqlCommand command = new MySqlCommand(query, connection))
        {
            command.Parameters.AddWithValue("@id_incident",
unTravailTechnicien.Id_incident);
            command.Parameters.AddWithValue("@id_technicien",
unTravailTechnicien.Id_technicien);
            command.Parameters.AddWithValue("@date_travail",
unTravailTechnicien.Date_travail);
            command.Parameters.AddWithValue("@heure_debut",
unTravailTechnicien.Heure_debut);
            command.Parameters.AddWithValue("@heure_fin",
unTravailTechnicien.Heure_fin);
            command.Parameters.AddWithValue("@travail_realise",
unTravailTechnicien.Travail_realise);
            command.ExecuteNonQuery();
        }
    }
  }

  public static void ModifierTechnicien(Technicien unTechnicien)
  {
    DatabaseConnection db = new DatabaseConnection();
```

```
    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();


        // --- Modifier la table personnel ---
        string queryPersonnel = "UPDATE personnel SET nom = @nom, prenom =
@prenom, matricule = @matricule, " +
                        "date_embauche = @date_embauche, mot_de_passe =
@mot_de_passe " +
                        "WHERE id_personnel = @id_personnel;";


        using (MySqlCommand cmd = new MySqlCommand(queryPersonnel,
connection))
        {
            cmd.Parameters.AddWithValue("@nom", unTechnicien.Nom);
            cmd.Parameters.AddWithValue("@prenom", unTechnicien.Prenom);
            cmd.Parameters.AddWithValue("@matricule", unTechnicien.Matricule);
            cmd.Parameters.AddWithValue("@date_embauche",
unTechnicien.Date_embauche);
            cmd.Parameters.AddWithValue("@mot_de_passe",
unTechnicien.Mot_de_passe);
            cmd.Parameters.AddWithValue("@id_personnel", unTechnicien.Id_personnel);


            cmd.ExecuteNonQuery();
        }


        // --- Modifier la table technicien ---
        string queryTech = "UPDATE technicien SET niveau_intervention = @niveau,
formation = @formation, " +
                    "competence = @competence, id_region = @id_region " +
```

```
                "WHERE id_personnel = @id_personnel;";

        using (MySqlCommand cmd2 = new MySqlCommand(queryTech, connection))
        {
            cmd2.Parameters.AddWithValue("@niveau",
unTechnicien.Niveau_intervention);
            cmd2.Parameters.AddWithValue("@formation", unTechnicien.Formation);
            cmd2.Parameters.AddWithValue("@competence", unTechnicien.Competence);
            cmd2.Parameters.AddWithValue("@id_region", unTechnicien.Id_region);
            cmd2.Parameters.AddWithValue("@id_personnel", unTechnicien.Id_personnel);

            cmd2.ExecuteNonQuery();
        }
    }
  }

  public static void ModifierVisiteur(Personnel unPersonnel)
  {
    DatabaseConnection db = new DatabaseConnection();

    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();

        string query = "UPDATE personnel SET nom = @nom, prenom = @prenom,
matricule = @matricule, " +
                "date_embauche = @date_embauche, mot_de_passe = @mot_de_passe "
+
                "WHERE id_personnel = @id_personnel;";
```

```
        using (MySqlCommand cmd = new MySqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("@nom", unPersonnel.Nom);
            cmd.Parameters.AddWithValue("@prenom", unPersonnel.Prenom);
            cmd.Parameters.AddWithValue("@matricule", unPersonnel.Matricule);
            cmd.Parameters.AddWithValue("@date_embauche",
unPersonnel.Date_embauche);
            cmd.Parameters.AddWithValue("@mot_de_passe",
unPersonnel.Mot_de_passe);
            cmd.Parameters.AddWithValue("@id_personnel", unPersonnel.Id_personnel);


            cmd.ExecuteNonQuery();
        }
    }
  }


  public static void ModifierMateriel(Materiel unMateriel)
  {
    DatabaseConnection db = new DatabaseConnection();


    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();


        string query = "UPDATE materiel SET ref_materiel = @ref_materiel,
carac_processeur = @processeur, " +
                    "carac_memoire = @memoire, carac_disque = @disque,
date_achat_location = @date_achat, " +
```

```
                "date_garantie = @garantie, fournisseur = @fournisseur,
id_personnel_affecte = @id_personnel " +
                "WHERE id_materiel = @id_materiel;";

        using (MySqlCommand cmd = new MySqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("@ref_materiel", unMateriel.Ref_materiel);
            cmd.Parameters.AddWithValue("@processeur", unMateriel.Processeur);
            cmd.Parameters.AddWithValue("@memoire", unMateriel.Memoir);
            cmd.Parameters.AddWithValue("@disque", unMateriel.Disque_dur);
            cmd.Parameters.AddWithValue("@date_achat", unMateriel.Date_achat);
            cmd.Parameters.AddWithValue("@garantie", unMateriel.Garantie);
            cmd.Parameters.AddWithValue("@fournisseur", unMateriel.Fournisseur);
            cmd.Parameters.AddWithValue("@id_personnel", unMateriel.Id_personnel);
            cmd.Parameters.AddWithValue("@id_materiel", unMateriel.Id_materiel);

            cmd.ExecuteNonQuery();
        }
    }
  }

  public static void ModifierIncident(Incident inc)
  {
    DatabaseConnection db = new DatabaseConnection();

    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
```

```
        string query = "UPDATE incident SET objet_incident = @objet, niveau_urgence =
@urgence, " +
                "date_declaration = @date_decl, id_declarant = @id_decl, id_materiel =
@id_mat, " +
                "id_etat = @etat, id_type_pc = @type_pc, id_technicien_en_charge =
@tech, " +
                "commentaire_responsable = @commentaire " +
                "WHERE id_incident = @id_incident;";


        using (MySqlCommand cmd = new MySqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("@objet", inc.Objet);
            cmd.Parameters.AddWithValue("@urgence", inc.Niveau_urgence);
            cmd.Parameters.AddWithValue("@date_decl", inc.Date_declaration);
            cmd.Parameters.AddWithValue("@id_decl", inc.Id_declarant);
            cmd.Parameters.AddWithValue("@id_mat", inc.Id_materiel);
            cmd.Parameters.AddWithValue("@etat", inc.Etat);
            cmd.Parameters.AddWithValue("@type_pc", inc.Id_type);
            cmd.Parameters.AddWithValue("@tech", inc.Id_technicien);
            cmd.Parameters.AddWithValue("@commentaire", "");
            cmd.Parameters.AddWithValue("@id_incident", inc.Id_incident);


            cmd.ExecuteNonQuery();
        }
    }
  }


  public static void ModifierTravailTechnicien(TravailTechnicien t)
```

```
    {
        DatabaseConnection db = new DatabaseConnection();

        using (MySqlConnection connection = db.GetConnection())
        {
            connection.Open();

            string query = "UPDATE travailtechnicien SET id_incident = @id_incident,
id_technicien = @id_technicien, " +
                    "date_travail = @date_travail, heure_debut = @heure_debut, heure_fin =
@heure_fin, " +
                    "travail_realise = @travail_realise WHERE id_travail = @id_travail;";

            using (MySqlCommand cmd = new MySqlCommand(query, connection))
            {
                cmd.Parameters.AddWithValue("@id_incident", t.Id_incident);
                cmd.Parameters.AddWithValue("@id_technicien", t.Id_technicien);
                cmd.Parameters.AddWithValue("@date_travail", t.Date_travail);
                cmd.Parameters.AddWithValue("@heure_debut", t.Heure_debut);
                cmd.Parameters.AddWithValue("@heure_fin", t.Heure_fin);
                cmd.Parameters.AddWithValue("@travail_realise", t.Travail_realise);
                cmd.Parameters.AddWithValue("@id_travail", t.Id_travail);

                cmd.ExecuteNonQuery();
            }
        }
    }

    public static void SupprimerTechnicien(int idPersonnel)
```

```
    {
        DatabaseConnection db = new DatabaseConnection();

        using (MySqlConnection connection = db.GetConnection())
        {
            connection.Open();

            // 1) Supprimer dans technicien
            string queryTech = "DELETE FROM technicien WHERE id_personnel =
@id_personnel;";
            using (MySqlCommand cmdTech = new MySqlCommand(queryTech, connection))
            {
                cmdTech.Parameters.AddWithValue("@id_personnel", idPersonnel);
                cmdTech.ExecuteNonQuery();
            }

            // 2) Supprimer dans personnel
            string queryPers = "DELETE FROM personnel WHERE id_personnel =
@id_personnel;";
            using (MySqlCommand cmdPers = new MySqlCommand(queryPers, connection))
            {
                cmdPers.Parameters.AddWithValue("@id_personnel", idPersonnel);
                cmdPers.ExecuteNonQuery();
            }
        }
    }

    public static void SupprimerVisiteur(int idPersonnel)
    {
```

```
    DatabaseConnection db = new DatabaseConnection();

    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();

        string query = "DELETE FROM personnel WHERE id_personnel = @id_personnel;";
        using (MySqlCommand cmd = new MySqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("@id_personnel", idPersonnel);
            cmd.ExecuteNonQuery();
        }
    }
}

public static void SupprimerMateriel(int idMateriel)
{
    DatabaseConnection db = new DatabaseConnection();

    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();

        string query = "DELETE FROM materiel WHERE id_materiel = @id_materiel;";
        using (MySqlCommand cmd = new MySqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("@id_materiel", idMateriel);
            cmd.ExecuteNonQuery();
```

```
        }
    }
}

public static void SupprimerIncident(int idIncident)
{
    DatabaseConnection db = new DatabaseConnection();

    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();

        string query = "DELETE FROM incident WHERE id_incident = @id_incident;";
        using (MySqlCommand cmd = new MySqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("@id_incident", idIncident);
            cmd.ExecuteNonQuery();
        }
    }
}

public static void SupprimerTravailTechnicien(int idTravail)
{
    DatabaseConnection db = new DatabaseConnection();

    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
```

```
        string query = "DELETE FROM travailtechnicien WHERE id_travail = @id_travail;";
        using (MySqlCommand cmd = new MySqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("@id_travail", idTravail);
            cmd.ExecuteNonQuery();
        }
    }
}

public static int Connexion(string matricule, string motDePasse)
{
    DatabaseConnection db = new DatabaseConnection();
    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
        string query = "SELECT id_fonction FROM personnel WHERE matricule =
@matricule AND mot_de_passe = @mot_de_passe;";
        using (MySqlCommand command = new MySqlCommand(query, connection))
        {
            command.Parameters.AddWithValue("@matricule", matricule);
            command.Parameters.AddWithValue("@mot_de_passe", motDePasse);
            object result = command.ExecuteScalar();
            if (result != null)
            {
                return Convert.ToInt32(result);
            }
            else
```

```
            {
                return -1; // Connexion échouée
            }
        }
    }
}

public static int GetPersonnelId(string matricule)
{
    DatabaseConnection db = new DatabaseConnection();
    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
        string query = "SELECT id_personnel FROM personnel WHERE matricule =
@matricule;";
        using (MySqlCommand command = new MySqlCommand(query, connection))
        {
            command.Parameters.AddWithValue("@matricule", matricule);
            object result = command.ExecuteScalar();
            if (result != null)
            {
                return Convert.ToInt32(result);
            }
            else
            {
                return -1; // Personnel non trouvé
            }
        }
```

```
    }
}

public static List<Materiel> GetAllMateriel()
{
    List<Materiel> materiels = new List<Materiel>();
    DatabaseConnection db = new DatabaseConnection();
    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
        string query = "SELECT * FROM materiel;";
        using (MySqlCommand command = new MySqlCommand(query, connection))
        {
            using (MySqlDataReader reader = command.ExecuteReader())
            {
                while (reader.Read())
                {
                    Materiel materiel = new Materiel(
                        reader.GetInt32("id_materiel"),
                        reader.GetString("ref_materiel"),
                        reader.GetString("carac_processeur"),
                        reader.GetString("carac_memoire"),
                        reader.GetString("carac_disque"),
                        reader.GetDateTime("date_achat_location"),
                        reader.GetDateTime("date_garantie"),
                        reader.GetString("fournisseur"),
                        reader.GetInt32("id_personnel_affecte")
                    );
```

```
                materiels.Add(materiel);
            }
        }
    }
}

    return materiels;
}

public static List<Personnel> GetAllVisiteur()
{
    List<Personnel> visiteurs = new List<Personnel>();
    DatabaseConnection db = new DatabaseConnection();
    using (MySqlConnection connection = db.GetConnection())
    {
        connection.Open();
        string query = "SELECT id_personnel, nom, prenom, matricule, date_embauche,
mot_de_passe, id_fonction FROM personnel WHERE id_fonction = 1;";
        using (MySqlCommand command = new MySqlCommand(query, connection))
        {
            using (MySqlDataReader reader = command.ExecuteReader())
            {
                while (reader.Read())
                {
                    Personnel visiteur = new Personnel(
                        reader.GetInt32("id_personnel"),
                        reader.GetString("nom"),
                        reader.GetString("prenom"),
```

```
                reader.GetString("matricule"),
                reader.GetDateTime("date_embauche"),
                reader.GetString("mot_de_passe"),
                reader.GetInt32("id_fonction")
            );
            visiteurs.Add(visiteur);
        }
    }
  }
  }
  return visiteurs;
}

public static List<Incident> GetAllIncident()
{
  List<Incident> incidents = new List<Incident>();
  DatabaseConnection db = new DatabaseConnection();
  using (MySqlConnection connection = db.GetConnection())
  {
    connection.Open();
    string query = "SELECT id_incident, objet_incident, niveau_urgence,
date_declaration, id_declarant, id_materiel, id_etat, id_type_pc,
id_technicien_en_charge FROM incident;";
    using (MySqlCommand command = new MySqlCommand(query, connection))
    {
      using (MySqlDataReader reader = command.ExecuteReader())
      {
        while (reader.Read())
        {
```

```
                Incident incident = new Incident(
                  reader.GetInt32("id_incident"),
                  reader.GetString("objet_incident"),
                  reader.GetInt32("niveau_urgence"),
                  reader.GetDateTime("date_declaration"),
                  reader.GetInt32("id_declarant"),
                  reader.GetInt32("id_materiel"),
                  reader.GetInt32("id_etat"),
                  reader.GetInt32("id_type_pc"),
                  reader.GetInt32("id_technicien_en_charge")
                );
                incidents.Add(incident);
              }
            }
          }
        }
        return incidents;
      }

    }
```

6.3 Form

LoginForm :

```
public partial class LoginForm : Form
{
    public LoginForm()
    {
        InitializeComponent();
    }

    private void Form1_Load(object sender, EventArgs e)
    {

    }

    private void buttonConexion_Click(object sender, EventArgs e)
    {
        string matricule = textBoxMatricule.Text;
        string mdp = textBoxMotDePasse.Text;
        int idUtilisateur =-1;

        int role = DatabaseConnection.Connexion(matricule, mdp);

        if (role > 0)
        {
            MessageBox.Show("Connexion réussie !");
            idUtilisateur = DatabaseConnection.GetPersonnelId(matricule);
        }

        switch (role)
        {
```

```
            case 1:
                // Visiteur
                VisiteurForm visiteurForm = new VisiteurForm(idUtilisateur);
                visiteurForm.Show();
                this.Hide();
                break;

            case 2:
                // Responsable
                ResponsableForm respForm = new ResponsableForm(idUtilisateur);
                respForm.Show();
                this.Hide();
                break;

            case 3:
                // Technicien
                TechnicienForm techForm = new TechnicienForm(idUtilisateur);
                techForm.Show();
                this.Hide();
                break;

            default:
                MessageBox.Show("Matricule ou mot de passe incorrect !");
                break;
        }
    }
}
```

VisiteurForm :

```
public partial class VisiteurForm : Form
{
    private int idUtilisateur;
    private List<Incident> ListIncident = new List<Incident>();
    private List<Materiel> ListMateriel = new List<Materiel>();

    public VisiteurForm(int id)
    {
        InitializeComponent();
        idUtilisateur = id;
    }

    private void VisiteurForm_Load(object sender, EventArgs e)
    {

        ListMateriel = DatabaseConnection.GetAllMateriel();
        ListIncident = DatabaseConnection.GetAllIncident();

        comboBoxMateriel.Items.Clear();
        comboBoxMateriel.DataSource = ListMateriel;
        comboBoxMateriel.DisplayMember = "Ref_materiel";
        comboBoxMateriel.ValueMember = "Id_materiel";

        comboBoxIncident.Items.Clear();
        comboBoxIncident.DataSource = ListIncident;
        comboBoxIncident.DisplayMember = "objet_incident";
```

```
        comboBoxIncident.ValueMember = "Id_incident";

        comboBoxNivUrgence.Items.Clear();
        comboBoxNivUrgence.DataSource = NivUrgence.Liste;
        comboBoxNivUrgence.DisplayMember = "libelle";
        comboBoxNivUrgence.ValueMember = "id";

        comboBoxTypePc.Items.Clear();
        comboBoxTypePc.DataSource = TypePc.Liste;
        comboBoxTypePc.DisplayMember = "libelle";
        comboBoxTypePc.ValueMember = "id_type";



    }

    private void buttonAjtIncident_Click(object sender, EventArgs e)
    {

        int newId = ListIncident.Count > 0 ? ListIncident.Max(i => i.Id_incident) + 1 : 1;

        Incident unIncident = new Incident(newId, textBoxDesc.Text,
(int)comboBoxNivUrgence.SelectedValue, DateTime.Now, idUtilisateur,
(int)comboBoxMateriel.SelectedValue, 1, (int)comboBoxTypePc.SelectedValue, null);
        DatabaseConnection.AjouterIncident(unIncident);
    }

    private void buttonConsulter_Click(object sender, EventArgs e)
    {
```

```
        List<Materiel> mesMateriels = Materiel.FilterByPersonnelId(ListMateriel, idUtilisateur);

        listBoxConsMate.Items.Clear();

        foreach (Materiel mat in mesMateriels)

        {

            listBoxConsMate.Items.Add("Référence : " + mat.Ref_materiel + ", Processeur : " + mat.Processeur + ", Mémoire : " + mat.Memoir + ", Disque Dur : " + mat.Disque_dur + ", Date d'achat : " + mat.Date_achat.ToShortDateString() + ", Garantie : " + mat.Garantie.ToShortDateString() + ", Fournisseur : " + mat.Fournisseur);

        }


    }


    private void buttonAffEtat_Click(object sender, EventArgs e)

    {

        int rang = comboBoxIncident.SelectedIndex;


        listBoxEtat.Items.Add("Etat de l'incident : " + ListIncident[rang].Etat);

    }

}
```

TechnicienForm :

```
public partial class ResponsableForm : Form

{

    private int idUtilisateur;

    private List<Personnel> lesVisiteurs = new List<Personnel>();
```

```
public ResponsableForm(int id)
{
    InitializeComponent();
    idUtilisateur = id;
    ChargerListes();
}

private void ChargerListes()
{
    lesVisiteurs = DatabaseConnection.GetAllVisiteur();

    comboBox5.DataSource = null;
    comboBox5.DataSource = lesVisiteurs;
    comboBox5.DisplayMember = "Nom";
    comboBox5.ValueMember = "Id_personnel";

    comboBox1.DataSource = null;
    comboBox1.DataSource = lesVisiteurs;
    comboBox1.DisplayMember = "Nom";
    comboBox1.ValueMember = "Id_personnel";
}

private void tabPage1_Click(object sender, EventArgs e)
{
}

private void button2_Click(object sender, EventArgs e)
{
```

```
try
{
    string nom = textBox1.Text;
    string prenom = textBox2.Text;
    string matricule = textBox3.Text;
    string mdp = textBox4.Text;
    DateTime dateEmbauche = dateTimePicker1.Value;

    if (radioButton1.Checked) // Technicien
    {

        Technicien tech = new Technicien(0, nom, prenom, matricule, dateEmbauche,
mdp, 2, 0, 1, "Formation par défaut", "Compétences par défaut", 1);
        DatabaseConnection.AjouterTechnicien(tech);
        MessageBox.Show("Technicien ajouté avec succès !");
    }
    else // Visiteur (radioButton2 ou défaut)
    {
        Personnel pers = new Personnel(0, nom, prenom, matricule, dateEmbauche,
mdp, 1);
        DatabaseConnection.AjouterVisiteur(pers);
        MessageBox.Show("Visiteur ajouté avec succès !");
    }
    ChargerListes();
}
catch (Exception ex)
{
    MessageBox.Show("Erreur lors de l'ajout : " + ex.Message);
}
```

```
}

private void button1_Click(object sender, EventArgs e)
{
    try
    {
        if (comboBox5.SelectedItem == null) return;

        int idSelect = (int)comboBox5.SelectedValue;

        Personnel p = new Personnel(
            idSelect,
            textBox8.Text,
            textBox7.Text,
            textBox6.Text,
            dateTimePicker2.Value,
            textBox5.Text,
            1 // Fonction visiteur par défaut pour l'exemple
        );

        DatabaseConnection.ModifierVisiteur(p);
        MessageBox.Show("Utilisateur modifié !");
        ChargerListes();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Erreur : " + ex.Message);
    }
```

```
    }

    private void button3_Click(object sender, EventArgs e)
    {
        try
        {
            if (comboBox1.SelectedItem != null)
            {
                int idToDelete = (int)comboBox1.SelectedValue;
                DatabaseConnection.SupprimerVisiteur(idToDelete);
                MessageBox.Show("Utilisateur supprimé.");
                ChargerListes();
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show("Erreur : " + ex.Message);
        }
    }
}
```

ResponsableForm :

```
public partial class TechnicienForm : Form
{
    private int idUtilisateur;
    private List<Materiel> lesMateriels;
```

```
private List<Incident> lesIncidents;

public TechnicienForm(int id)
{
    InitializeComponent();
    idUtilisateur = id;
    ChargerDonnees();
}

private void ChargerDonnees()
{
    lesMateriels = DatabaseConnection.GetAllMateriel();
    lesIncidents = DatabaseConnection.GetAllIncident();

    comboBox5.DataSource = null;
    comboBox5.DataSource = lesMateriels;
    comboBox5.DisplayMember = "Ref_materiel";

    comboBox6.DataSource = null; // Supprimer matériel
    comboBox6.DataSource = lesMateriels;
    comboBox6.DisplayMember = "Ref_materiel";
    comboBox6.ValueMember = "Id_materiel";

    comboBox7.DataSource = null;
    comboBox7.DataSource = lesIncidents;
    comboBox7.DisplayMember = "Objet";

    comboBox8.DataSource = null;
```

```
    comboBox8.DataSource = lesIncidents;
    comboBox8.DisplayMember = "Objet";
    comboBox8.ValueMember = "Id_incident";
}

private void tabPage1_Click(object sender, EventArgs e)
{
}



private void button2_Click(object sender, EventArgs e)
{
    try
    {

        Materiel m = new Materiel(
            0,
            textBox1.Text, // Ref
            textBox3.Text, // Processeur
            textBox4.Text, // Mémoire
            textBox5.Text, // Disque
            dateTimePicker1.Value, // Date achat
            dateTimePicker4.Value, // Garantie
            "Fournisseur X", // Pas de champ visible, valeur par défaut
            idUtilisateur // Affecté à soi-même ou null
        );

        DatabaseConnection.AjouterMateriel(m);
```

```
            MessageBox.Show("Matériel ajouté !");
            ChargerDonnees();
        }
        catch (Exception ex)
        {
            MessageBox.Show("Erreur ajout : " + ex.Message);
        }
    }



    private void button3_Click(object sender, EventArgs e)
    {
        if (comboBox5.SelectedItem != null)
        {
            Materiel m = (Materiel)comboBox5.SelectedItem;
            richTextBox1.Text = $"Ref: {m.Ref_materiel}\nProc: {m.Processeur}\nRAM:
{m.Memoir}\nHDD: {m.Disque_dur}\nGarantie: {m.Garantie}";
        }
    }



    private void button4_Click(object sender, EventArgs e)
    {
        if (comboBox6.SelectedValue != null)
        {
            DatabaseConnection.SupprimerMateriel((int)comboBox6.SelectedValue);
            MessageBox.Show("Matériel supprimé.");
            ChargerDonnees();
```

```
        }
    }



    // BOUTON CONSULTER INCIDENT (button6)
    private void button6_Click(object sender, EventArgs e)
    {
        listBox1.Items.Clear();
        foreach (var inc in lesIncidents)
        {
            listBox1.Items.Add($"{inc.Id_incident} - {inc.Objet} (Urgence:
{inc.Niveau_urgence})");
        }
    }

    private void button8_Click(object sender, EventArgs e)
    {
        try
        {
            if (comboBox8.SelectedItem == null) return;

            Incident inc = (Incident)comboBox8.SelectedItem;
            inc.Etat = 2;
            inc.Id_technicien = idUtilisateur;

            DatabaseConnection.ModifierIncident(inc);
            TravailTechnicien travail = new TravailTechnicien(
                0,
```

```
            inc.Id_incident,
            idUtilisateur,
            DateTime.Now,
            DateTime.Now.TimeOfDay,
            DateTime.Now.TimeOfDay,
            textBox8.Text // Commentaire/Travail réalisé
        );

        DatabaseConnection.AjouterTravailTechnicien(travail);

        MessageBox.Show("Incident pris en charge !");
    }
    catch (Exception ex)
    {
        MessageBox.Show("Erreur : " + ex.Message);
    }
  }
  private void button9_Click(object sender, EventArgs e)
  {
    MessageBox.Show("Fonctionnalité mise à jour.");
  }
}
```